Despachador: objetos de base de datos para quitar las consultas directas de los DAO

Zaca - Terminal de autobuses | 27 de octubre de 2026 | Alcance: solo la seccion del despachador (Programar salida, Manifiesto y Abordaje), contrastado con la base real **terminal** (PostgreSQL 16.9).

1. Resumen

Los DAO del despachador son tres. **AbordajeDAO** y **ManifiestoDAO** ya llaman objetos de la base (5 metodos) y esos objetos **ya existen** en la base real. Lo que sigue con SQL de tablas dentro de Java es **SalidaDAO**: 6 sentencias (5 consultas y 1 INSERT) que usan las tres pantallas, porque Manifiesto y Abordaje cargan su selector de salidas con `SalidaDAO.listar()`. Se mantiene la regla del contrato: **vista** para consultas armadas, **funcion** para lecturas con parametros, **procedure** para lo que modifica datos, **trigger** para validar. El DAO solo hace SELECT sobre una vista o funcion, o CALL a un procedure.

Concepto	Cantidad	Detalle
Metodos de DAO revisados	11	AbordajeDAO 2, ManifiestoDAO 3, SalidaDAO 6.
Ya resueltos con objetos existentes	5 metodos	Abordaje y Manifiesto usan 7 objetos que existen en la base (se comprobaron uno por uno). Ver seccion 2.
Se resuelve con una vista que ya existe	1 metodo	`SalidaDAO.rutas()` puede leer `viajes.v_rutas`, que ya existe. No hace falta crear nada.
Objetos nuevos propuestos	5	2 vistas, 2 funciones y 1 procedure. Los 5 se crearon en una transaccion de prueba con datos de mentira y se hizo ROLLBACK (seccion 7).
Triggers nuevos	0	Las reglas de programar una salida ya las valida `viajes.fn_asignar_conductor`. Hay 3 decisiones abiertas sobre reglas faltantes (seccion 8).

Convenciones

Las mismas del documento del administrador: vistas **v_***, funciones **fn_***, procedures **sp_***; esquemas **administracion** y **viajes**. Los errores de regla salen con **RAISE EXCEPTION ... USING ERRCODE** y Java los lee con `e.getSQLState()` (`Mensaje.deViaje` ya traduce Z0006, Z0007 y Z0012). Cada DAO lee el ResultSet **por posicion**, asi que las columnas deben salir en el orden de las tablas de abajo.

2. Inventario de los DAO del despachador

DAO	Metodo	Consulta de hoy	Objeto	Estado en la base
AbordajeDAO	boletos(folio)	SELECT id_boleto, id_viaje, num_asiento, estado_boleto FROM fn_buscar_boletos_por_folio(?)	`ventas.fn_buscar_boletos_por_folio` (usa `ventas.v_boletos_abordaje`)	Ya existe
AbordajeDAO	abordar(ids)	CALL ventas.sp_marcar_abordado(?) por cada boleto, en una transaccion	`ventas.sp_marcar_abordado`	Ya existe
ManifiestoDAO	escalas(salida)	SELECT ... FROM fn_obtener_escalas(?)	`viajes.fn_obtener_escalas`	Ya existe
ManifiestoDAO	pasajeros(viaje, terminal)	SELECT ... FROM fn_buscar_pasajeros_manifiesto(?, ?)	`ventas.fn_buscar_pasajeros_manifiesto` (usa `ventas.v_manifiesto_detallado`)	Ya existe
ManifiestoDAO	marcarEnRuta(viaje)	CALL viajes.sp_marcar_viaje_en_ruta(?)	`viajes.sp_marcar_viaje_en_ruta`	Ya existe
SalidaDAO	rutas()	SELECT viajes.ruta ORDER BY nombre_ruta	`viajes.v_rutas`	Existe: solo cambiar el DAO
SalidaDAO	unidades()	SELECT unidad + tipo_bus con texto armado 'placa · modelo · N asientos'	`viajes.v_unidades_programables`	**NUEVA**
SalidaDAO	choferes()	SELECT personal + puesto WHERE puesto = 'CONDUCTOR'	`administracion.v_conductores`	**NUEVA**
SalidaDAO	listar(dia)	SELECT viaje + ruta + servicio + estado + unidad + tipo_bus + personal (7 tablas) con rango de fechas	`viajes.fn_salidas_del_dia`	**NUEVA**
SalidaDAO	programar(...)	INSERT viaje_programado con subconsulta del estado PROGRAMADO	`viajes.sp_programar_viaje`	**NUEVO**
SalidaDAO	recorrido(salida)	SELECT ruta_paradas + terminal + 3 subconsultas de conteo sobre boleto	`viajes.fn_recorrido_viaje`	**NUEVA**

Nota sobre `viajes.cartelera_viajes`**:** existe en la base, pero no sirve para `listar()`: no trae `id_ruta`, `capacidad` ni la vigencia de la licencia del chofer, que el codigo si usa (el detalle de la salida y el aviso de licencia). Por eso se propone una funcion nueva en vez de reutilizarla.

3. Lo que se encontro en la base real

Se consulto el esquema vivo (funciones, procedures, vistas, triggers, constraints, extensiones y roles). Esto precisa o corrige lo que se asumia.

Hallazgo	Efecto en el diseno
Los 7 objetos de Abordaje y Manifiesto ya existen	Estan creados en la base y coinciden con lo que llaman los DAO: `v_boletos_abordaje`, `fn_buscar_boletos_por_folio`, `sp_marcar_abordado`, `fn_obtener_escalas`, `v_manifiesto_detallado`, `fn_buscar_pasajeros_manifiesto` y `sp_marcar_viaje_en_ruta`.
Programar una salida ya se valida con un trigger	`tigresito_validad_conductor` (BEFORE INSERT/UPDATE en viaje_programado) llama a `viajes.fn_asignar_conductor`: Z0007 si no es CONDUCTOR, Z0006 si no hay licencia o esta vencida el dia del viaje, Z0012 si la unidad o el chofer ya tienen otro viaje que se empalme (considera la duracion de la ruta). **Por eso** `sp_programar_viaje` **solo hace el INSERT**: no repite validaciones.
Z0006 ya funciona	En la revision anterior el bloque de Z0006 estaba vacio; ahora la funcion si lanza el error y se comprobo (seccion 7).
Los ids son GENERATED ALWAYS AS IDENTITY	El INSERT de viaje no manda id. Java no necesita el id nuevo, asi que `sp_programar_viaje` no lleva parametro INOUT.
Los catalogos de estado se buscan por nombre	Estados de viaje: 1 PROGRAMADO, 2 ABORDANDO, 3 EN RUTA, 4 CERRADO, 5 SIN UNIDAD. El procedure busca PROGRAMADO por nombre, como hoy lo hace Java, no por el numero.
La aplicacion se conecta como **postgres**	Es superusuario, igual que se vio con el administrador. Cuando exista el rol de la aplicacion (por ejemplo zaca_app) con solo SELECT en vistas y EXECUTE en procedures/funciones, `sp_programar_viaje` **debera ser SECURITY DEFINER con SET search_path fijo**, o el INSERT fallara por permisos. Los 5 objetos nuevos deben recibir ese GRANT.
No hay FDW todavia	La base **terminal** solo tiene las extensiones plpgsql y btree_gist (no postgres_fdw), no hay servidores foraneos y en el cluster no existe una base del taller. El bloqueo de unidades en mantenimiento aun no se puede hacer en la base. Ver decision 1.
Una consulta con parametros no puede llevar subconsultas en un CALL	PostgreSQL rechaza `CALL sp_x((SELECT ...))`. En Java no afecta (se mandan valores con ?), pero hay que tenerlo presente al probar a mano.
Datos hoy	La base tiene 0 viajes, 0 boletos, 1 unidad y 1 ruta; la prueba de la seccion 7 uso datos de mentira dentro de una transaccion.

4. Vistas

Regla para que no falle: los DAO leen el ResultSet por posicion, no por nombre. Las dos columnas de cada catalogo son id (integer) y texto, que es lo que lee `Sql.catalogo()` con `getInt(1)` y `getString(2)`. Una vista no garantiza el orden de las filas: cada DAO conserva su ORDER BY al hacer el SELECT.

`viajes.v_unidades_programables (NUEVA)`

Combo Unidad de Programar salida. Reemplaza `SalidaDAO.unidades()`. El DAO hace `SELECT id_unidad, descripcion FROM viajes.v_unidades_programables ORDER BY descripcion`.

Pos	Columna	Tipo	NULL	Como la lee Java
1	`id_unidad`	integer	NO	getInt(1) -> Catalogo.id
2	`descripcion`	text	NO	getString(2) -> Catalogo.nombre. Texto ya armado: 'placa · modelo · N asientos'

`administracion.v_conductores (NUEVA)`

Combo Chofer de Programar salida: solo personal con puesto CONDUCTOR. Reemplaza `SalidaDAO.choferes()`. El DAO hace `SELECT id_personal, nombre_completo FROM administracion.v_conductores ORDER BY nombre_completo`.

Pos	Columna	Tipo	NULL	Como la lee Java
1	`id_personal`	integer	NO	getInt(1) -> Catalogo.id. Es el id_conductor que recibe sp_programar_viaje
2	`nombre_completo`	text	NO	getString(2) -> Catalogo.nombre. nombre + ap_paterno + ap_materno (el materno puede ser NULL, se omite)

`viajes.v_rutas (YA EXISTE)`

Combo Ruta. Reemplaza `SalidaDAO.rutas()` sin crear nada: el DAO hace `SELECT id_ruta, nombre_ruta FROM viajes.v_rutas ORDER BY nombre_ruta`. La vista tiene 4 columnas; el catalogo lee solo las dos primeras.

Pos	Columna	Tipo	NULL	Como la lee Java
1	`id_ruta`	integer	NO	getInt(1) -> Catalogo.id
2	`nombre_ruta`	text	NO	getString(2) -> Catalogo.nombre
3	`id_servicio, nombre_servicio (extra)`	integer, text	NO	No se leen en esta pantalla

5. Funciones (2): columnas exactas

`viajes.fn_salidas_del_dia(p_dia DATE) (NUEVA)`

Reemplaza `SalidaDAO.listar(dia)`, que usa Programar salida y tambien los selectores de Manifiesto y Abordaje. Llamada: `SELECT * FROM viajes.fn_salidas_del_dia(?)` con `ps.setObject(1, dia)` (LocalDate). Devuelve los viajes cuya fecha de salida, en hora de Mexico (America/Mexico_City), cae en ese dia, ordenados por hora. Equivale al rango de fechas que hoy calcula Java con `Formato.ZONA`.

Pos	Columna	Tipo	NULL	Como la lee Java
1	`id_viaje`	integer	NO	getInt(1) -> Salida.idViaje
2	`id_ruta`	integer	NO	getInt(2) -> Salida.idRuta (la usa Manifiesto para pedir las escalas)
3	`nombre_ruta`	text	NO	getString(3) -> Salida.ruta
4	`nombre_servicio`	text	NO	getString(4) -> Salida.servicio
5	`fecha_salida`	timestamptz	NO	getObject(5, OffsetDateTime.class).atZoneSameInstant(Formato.ZONA) -> Salida.fecha
6	`nombre_estado_viaje`	text	NO	getString(6) -> Salida.estado
7	`placa`	text	NO	getString(7) -> Salida.placa
8	`modelo`	text	NO	getString(8) -> Salida.modelo
9	`capacidad`	integer	NO	getInt(9) -> Salida.capacidad (se usa en la ocupacion por tramo)
10	`conductor`	text	NO	getString(10) -> Salida.chofer. nombre + ap_paterno + ap_materno (el materno puede ser NULL)
11	`vigencia`	date	SI	getObject(11, LocalDate.class) -> Salida.vigenciaLicencia (NULL si el conductor no tiene vigencia capturada)

`viajes.fn_recorrido_viaje(p_id_viaje INT) (NUEVA)`

Reemplaza `SalidaDAO.recorrido(salida)`, usado por la ventana Detalle de la salida. Llamada: `SELECT * FROM viajes.fn_recorrido_viaje(?)` con `ps.setInt(1, salida.idViaje())`. Ya no hace falta mandar el id de la ruta: la funcion la toma del viaje. Una fila por parada, ordenadas por `orden`. Los conteos son **integer** (la funcion convierte el count), no bigint. No cuentan boletos cancelados (id_estado_boleto 4).

Pos	Columna	Tipo	NULL	Como la lee Java
1	`nombre_terminal`	text	NO	getString(1) -> ParadaViaje.terminal
2	`min_desde_origen`	integer	NO	getInt(2): Java suma estos minutos a la hora de salida -> ParadaViaje.hora
3	`suben`	integer	NO	getInt(3) -> boletos con esa terminal de origen
4	`bajan`	integer	NO	getInt(4) -> boletos con esa terminal de destino
5	`ocupados_hacia_siguiente`	integer	NO	getInt(5) -> boletos cuyo tramo_ocupacion contiene el orden de la parada. En la ultima parada es 0

6. Procedure (1): parametros exactos

`viajes.sp_programar_viaje (NUEVO)`

Reemplaza el INSERT de `SalidaDAO.programar()`. Es **LANGUAGE plpgsql**; sin INOUT porque Java no usa el id nuevo. Inserta el viaje con estado PROGRAMADO (buscado por nombre). Las validaciones las hace el trigger existente al ejecutarse el INSERT.

#	Parametro	Tipo	NULL	Valor que manda Java
1	`p_id_ruta`	integer	NO	cbRuta.getValue().id()
2	`p_id_unidad`	integer	NO	cbUnidad.getValue().id()
3	`p_id_conductor`	integer	NO	cbChofer.getValue().id()
4	`p_fecha_salida`	timestamptz	NO	salida.toOffsetDateTime() con setObject. Hoy el DAO ya lo manda asi

Llamada desde Java (mismo patron que ya usan los otros DAO): `Sql.ejecutar(con, "CALL viajes.sp_programar_viaje(?, ?, ?, ?)", idRuta, idUnidad, idChofer, salida.toOffsetDateTime())` dentro de `Sql.transaccion`. No lleva INOUT, asi que no hace falta executeQuery.

Errores que llegan a Java (los lanza el trigger, ya traducidos por Mensaje.deViaje)

SQLState	Cuando	Mensaje en pantalla
`Z0007`	El empleado no tiene puesto CONDUCTOR (no deberia pasar: el combo ya filtra por v_conductores).	Esa persona no es conductor.
`Z0006`	El conductor no tiene licencia o su vigencia es anterior al dia de la salida (en hora de Mexico).	La licencia del chofer no esta vigente el dia de la salida.
`Z0012`	La unidad o el chofer ya tienen otro viaje PROGRAMADO, ABORDANDO o EN RUTA que se empalma con la duracion de este (la duracion es el mayor min_desde_origen de la ruta).	La unidad o el chofer ya tienen otro viaje en ese horario.
`23503`	Ruta, unidad o conductor inexistentes (clave foranea). No pasa si los combos salen de las vistas.	Mensaje generico de Mensaje.deViaje.

7. Pruebas realizadas contra la base real

Se creo cada objeto dentro de una transaccion (BEGIN ... ROLLBACK), con dos conductores de mentira (uno con licencia vigente y otro vencida). Al terminar se verifico que no quedo nada: 0 viajes y ninguna de las dos vistas creada. La base no se modifico.

Prueba	Resultado	Conclusion
Crear las 2 vistas, las 2 funciones y el procedure	CREATE VIEW x2, CREATE FUNCTION x2, CREATE PROCEDURE	La sintaxis y los tipos son validos en PostgreSQL 16.9.
v_unidades_programables	1 fila: 123456 · nidose · 60 asientos	Texto armado listo para el combo.
v_conductores	2 filas: 'Prueba Vencida Dos' y 'Prueba Vigente Uno'	Solo puesto CONDUCTOR; se ordena por apellidos.
CALL sp_programar_viaje (manana 10:00, conductor vigente)	CALL correcto	El viaje queda PROGRAMADO.
fn_salidas_del_dia(manana)	1 fila con las 11 columnas; fecha 2026-10-28 10:00-06, capacidad 60, vigencia 2027-02-04	Las posiciones 1 a 11 coinciden con el constructor de Salida.
fn_salidas_del_dia(hoy)	0 filas	El filtro por dia en hora de Mexico funciona.
fn_recorrido_viaje(viaje nuevo)	2 filas: PIJIJIAPAN (min 0) y TAPACHULA (min 120), todos los conteos en 0	Orden de las paradas y tipos correctos; sin boletos los conteos son 0.
Segundo viaje a las 10:30 con la misma unidad y chofer	ERROR Z0012: La unidad o el conductor ya tienen otro viaje en ese horario.	El trigger valida el empalme y el procedure no necesita repetirlo.
Viaje con el conductor de licencia vencida	ERROR Z0006: La licencia del conductor ... no esta vigente el dia del viaje.	Z0006 ya funciona.
Despues del ROLLBACK	0 viajes y 0 vistas nuevas	No quedo ningun dato de prueba.

No se pudo probar con datos reales el conteo de suben, bajan y ocupados de `fn_recorrido_viaje`: la base no tiene boletos. La logica es la misma de las tres subconsultas que hoy usa `SalidaDAO.recorrido()`, solo cambia que parte del id del viaje y que el resultado se convierte a integer.

8. Decisiones y discrepancias que el equipo debe resolver

#	Tema	Situacion	Que hay que decidir o hacer
1	Unidades en taller (FDW)	El proyecto pide bloquear la asignacion de unidades en mantenimiento. Hoy no hay FDW ni base de taller en el cluster, y `v_unidades_programables` lista todas las unidades. El modulo del taller ya tiene `v_semaforo_unidades` (placa, estatus, apto_para_viajar).	Decidir donde se bloquea. Propuesta: cuando exista el FDW, (a) agregar a la vista una columna `apto_para_viajar` al final para que Java solo muestre las aptas, y (b) agregar un trigger que rechace en la base una unidad no apta con un SQLState nuevo (por ejemplo Z0017) y su mensaje en Mensaje.deViaje. Con solo (a) cualquiera podria saltarse el bloqueo; con solo (b) el combo ofreceria unidades que luego se rechazan.
2	sp_marcar_abordado no valida el estado del boleto	El procedure existente pasa cualquier boleto a ABORDADO, aun si esta CANCELADO o ya ABORDADO. Hoy solo lo impide AbordajeController (en Java).	Decidir si el procedure debe rechazar los boletos que no esten VIGENTE (o tambien RESERVADO) con un error propio. Si no se hace, un cambio en Java podria marcar boletos cancelados como abordados.
3	sp_marcar_viaje_en_ruta no valida el estado del viaje	Pasa a EN RUTA cualquier viaje, aunque ya este CERRADO o EN RUTA. Hoy Java solo ofrece el boton cuando corresponde.	Decidir si el procedure solo acepta viajes PROGRAMADO o ABORDANDO, y que error lanza si no. Tambien queda abierto quien pasa el viaje a ABORDANDO: ningun objeto ni pantalla lo hace hoy.
4	Acentos dañados en un mensaje de la base	El texto de Z0006 dentro de `fn_asignar_conductor` tiene caracteres corruptos ('no est vigente el d¡a del viaje'). La pantalla no se ve afectada porque Java traduce por SQLState, pero el mensaje original aparece asi en logs y psql.	Volver a crear la funcion con el archivo guardado en UTF-8 (el .txt del script).
5	Permisos	La aplicacion usa el superusuario postgres, asi que un REVOKE no tendria efecto.	Crear el rol de la aplicacion y darle SELECT sobre las vistas y EXECUTE sobre funciones y procedures (incluyendo los 5 nuevos). `sp_programar_viaje` debe ser SECURITY DEFINER con search_path fijo.
6	Fecha de la salida y zona horaria	Se usa America/Mexico_City, igual que el trigger y Formato.ZONA. Si algun dia la base cambia de zona, `fn_salidas_del_dia` y Java deben cambiar juntos.	Confirmar que America/Mexico_City es la zona oficial del proyecto.

Orden sugerido para implementarlo

Paso	Que se hace	Archivos de Java que cambian
1	Crear en la base las 2 vistas, las 2 funciones y el procedure (el SQL probado esta en el apendice).	Ninguno
2	Cambiar `SalidaDAO`: rutas, unidades y choferes a las vistas; listar y recorrido a las funciones; programar al CALL.	SalidaDAO.java
3	Probar las tres pantallas del despachador: el selector de Manifiesto y Abordaje depende de listar().	Ninguno (los controllers no cambian)
4	Resolver las decisiones 2 y 3 (validaciones de estado) y, cuando exista el FDW, la decision 1.	Procedures y vista; Mensaje.deViaje si hay codigos nuevos

Apendice: SQL probado (listo para copiar)

Es el SQL que se ejecuto en la transaccion de prueba. Sigue el estilo del script (mayusculas, INNER JOIN, sin sangria).

```
CREATE OR REPLACE VIEW viajes.v_unidades_programables AS
SELECT u.id_unidad,
u.placa || ' · ' || u.modelo || ' · ' || tb.capacidad || ' asientos' AS descripcion
FROM viajes.unidad u
INNER JOIN viajes.tipo_bus tb ON tb.id_tipo_bus = u.id_tipo_bus
ORDER BY u.placa;


CREATE OR REPLACE VIEW administracion.v_conductores AS
SELECT p.id_personal,
p.nombre_personal || ' ' || p.ap_paterno || COALESCE(' ' || p.ap_materno, '') AS nombre_completo
FROM administracion.personal p
INNER JOIN administracion.puesto pu ON pu.id_puesto = p.id_puesto
WHERE pu.nombre_puesto = 'CONDUCTOR'
ORDER BY p.ap_paterno, p.ap_materno, p.nombre_personal;


CREATE OR REPLACE FUNCTION viajes.fn_salidas_del_dia(p_dia DATE)
RETURNS TABLE (
id_viaje INT, id_ruta INT, nombre_ruta TEXT, nombre_servicio TEXT, fecha_salida TIMESTAMPTZ,
nombre_estado_viaje TEXT, placa TEXT, modelo TEXT, capacidad INT, conductor TEXT, vigencia DATE
)
LANGUAGE sql
AS $$
SELECT v.id_viaje, r.id_ruta, r.nombre_ruta, ts.nombre_servicio, v.fecha_salida, ev.nombre_estado_viaje,
u.placa, u.modelo, tb.capacidad,
p.nombre_personal || ' ' || p.ap_paterno || COALESCE(' ' || p.ap_materno, ''), p.vigencia
FROM viajes.viaje_programado v
INNER JOIN viajes.ruta r ON r.id_ruta = v.id_ruta
INNER JOIN viajes.tipo_servicio ts ON ts.id_servicio = r.id_servicio
INNER JOIN viajes.estado_viaje ev ON ev.id_estado_viaje = v.id_estado_viaje
INNER JOIN viajes.unidad u ON u.id_unidad = v.id_unidad
INNER JOIN viajes.tipo_bus tb ON tb.id_tipo_bus = u.id_tipo_bus
INNER JOIN administracion.personal p ON p.id_personal = v.id_conductor
WHERE (v.fecha_salida AT TIME ZONE 'America/Mexico_City')::date = p_dia
ORDER BY v.fecha_salida;
$$;


CREATE OR REPLACE FUNCTION viajes.fn_recorrido_viaje(p_id_viaje INT)
RETURNS TABLE (nombre_terminal TEXT, min_desde_origen INT, suben INT, bajan INT, ocupados_hacia_siguiente INT)
LANGUAGE sql
AS $$
SELECT t.nombre_terminal, rp.min_desde_origen,
(SELECT count(*) FROM ventas.boleto b
 WHERE b.id_viaje = vp.id_viaje AND b.id_estado_boleto <> 4 AND b.id_terminal_origen = rp.id_terminal)::INT,
(SELECT count(*) FROM ventas.boleto b
 WHERE b.id_viaje = vp.id_viaje AND b.id_estado_boleto <> 4 AND b.id_terminal_destino = rp.id_terminal)::INT,
(SELECT count(*) FROM ventas.boleto b
 WHERE b.id_viaje = vp.id_viaje AND b.id_estado_boleto <> 4 AND b.tramo_ocupacion @> rp.orden)::INT
FROM viajes.viaje_programado vp
```

```
INNER JOIN viajes.ruta_paradas rp ON rp.id_ruta = vp.id_ruta
INNER JOIN administracion.terminal t ON t.id_terminal = rp.id_terminal
WHERE vp.id_viaje = p_id_viaje
ORDER BY rp.orden;
$$;


CREATE OR REPLACE PROCEDURE viajes.sp_programar_viaje(
p_id_ruta INT, p_id_unidad INT, p_id_conductor INT, p_fecha_salida TIMESTAMPTZ
)
LANGUAGE plpgsql
AS $$
BEGIN
INSERT INTO viajes.viaje_programado (id_ruta, id_estado_viaje, id_unidad, id_conductor, fecha_salida)
VALUES (p_id_ruta, (SELECT id_estado_viaje FROM viajes.estado_viaje WHERE nombre_estado_viaje = 'PROGRAMADO'),
p_id_unidad, p_id_conductor, p_fecha_salida);
END;
$$;
```